



[BIT703 Web Technologies](#) / [Contents](#) / Topic 7: Client-side web application development and programming – JavaScript / Understanding JavaScript

JavaScript objects 10m

Like other programming languages, JavaScript uses objects. Objects in programming can be a collection of properties, such as **variables**, **functions** and **data structures**. The purpose of objects is to effectively combine data and operations together, whereas functions only pass data back and forth.

JavaScript objects can be seen as simply a collection of properties that are grouped together. Properties can be added to, or removed from, an object. The values of these properties can be values of any JavaScript data type, including other objects. Each property in an object has a key as well as a value. In other words, entities are coded as **objects**. Each entity will group a given set of information (**properties**) and actions (**methods**) that can be performed by the entity.

A JavaScript object is like a real-world entity which has information (properties) and action (methods). For example, take a car. We can think about a car as an **object**. It will have **properties** like colour and model. Then the value of these properties (colour and model) are the **property value**. For example, the property value could be blue. It will also have **actions** like accelerating and braking.

JavaScript object syntax

JavaScript objects are enclosed within a pair of curly braces { }. Each property of the object has a key value (followed by a colon), and then the property value is defined. Each key/value pair is delimited by a comma.

In this example, an object called customer is created with three properties. This is an example of an object that looks like JSON code.

```
var customer = {firstName: "John", lastName: "Snow", height:1.82};
```

Objects can also contain other objects. For example, the above example can be written to include a name object that contains the firstName and lastName name strings.

```
const customer = { name: {firstName: "John", lastName: "Snow" }, height: 1.82 };
```

Object-oriented programming (OOP) is a programming paradigm that relies heavily on the concept of **objects**. It is used to structure a software programme into simple, reusable pieces of code blocks (classes), which can then be used to create individual instances of objects. OOP is very useful on large-scale projects as it facilitates code modularity and organising the objects.

Working With Objects (MDN, n.d.) is an excellent discourse in JavaScript objects from MDN web docs, and it is worth reading through should you wish to become more familiar with how JavaScript handles OOP.

You will find the use of OOP becomes more relevant as you work through the next section on the Document Object Model within the web browser.



☒ Objects

Non-Assessed

Status: Done

Reset ↺

[Skip to main content](#)

Have a look at the code below. What will the outcome of this code be when you run it.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>Any changes to a copy of an object will also change the original object:</p>

<p id="demo"></p>

<script>
//Object person with 4 properties
const person = {
  firstName: "John",
  lastName: "Doe",
  age:50,
  eyeColor: "blue"
};

const x = person;
x.age = 60;

document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
</script>

</body>
</html>
```

☐ John is 50 years old

☒ John is 60 years old



The code in this activity demonstrates that JavaScript objects are addressed by reference, not by value. If person is an object, then the following statement will not create a copy of person.

1. You might have thought that the code would execute the text 'John is 50 years old', but the code actually executed the text 'John is 60 years old'.

[Skip to main content](#)

2. The reason for this is that object x is not a copy of person. It is person. Both x and person are the same object. Any changes to x will also change person, because x and person are the same object.

Previous attempts ▼



Dig deeper

To learn more about JavaScript objects, read this resource which explain JavaScript objects and demonstrate how to use objects through examples:

- *JavaScript Arrays and Objects are Just Like Books and Newspapers* (Kononenko, 2017).



Thoughts on this page?

